

≡ OOPS CONCEPTS ≡

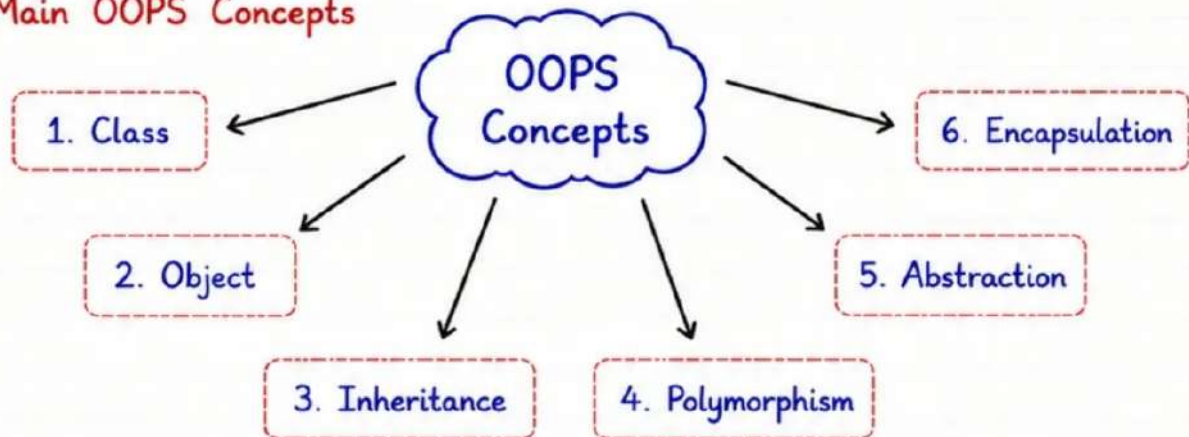
★ What is OOPS ?

OOPS (Object-Oriented Programming System) is a programming paradigm based on the concept of "Objects", which can contain data in the form of fields (attributes) and code in the form of procedures (methods).

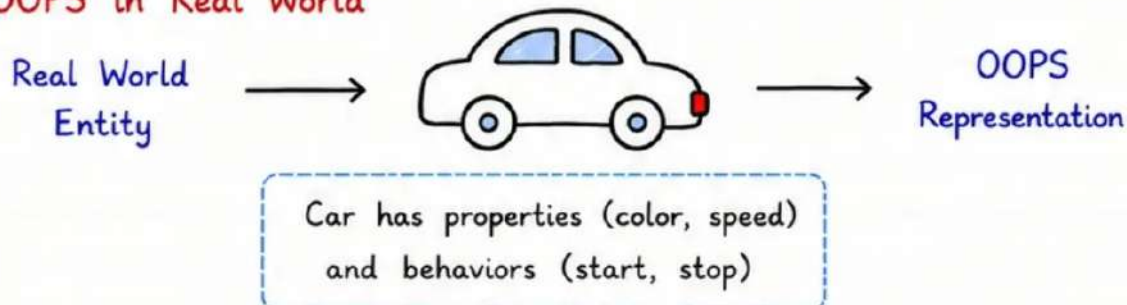
★ Why OOPS ?

- It makes programs easier to understand, maintain and modify.
- It provides data security by data hiding.
- It supports reusability through inheritance.
- It allows flexibility through polymorphism.
- It helps in modeling real-world entities.

★ Main OOPS Concepts



★ OOPS in Real World



★ Summary

OOPS is a powerful programming paradigm that uses objects and classes to design modular, reusable and scalable applications. The above concepts work together to make the code efficient and easy to manage.

Remember:

Think Classes,
Create Objects,
Build Solutions! ★

★ ≥ OOPS CONCEPT FULL HANDBOOK ≤ ★

≥ 1. CLASS ≤

★ What is a Class ?

A class is a **blueprint** or **template** for creating objects.
It defines a set of **attributes** (data) and **methods** (functions) that the objects of the class will have.

★ Key Points

- A class does **not** occupy memory.
- It is a **user-defined** data type.
- It acts as a **blueprint** for creating objects.
- It helps in **data hiding** and **code reusability**.

★ Syntax (in Java)

```
class ClassName {
    // Attributes
    dataType attribute1;
    dataType attribute2;
    ...
    // Methods
    returnType methodName(parameters) {
        // method body
    }
}
```

→ Class Keyword

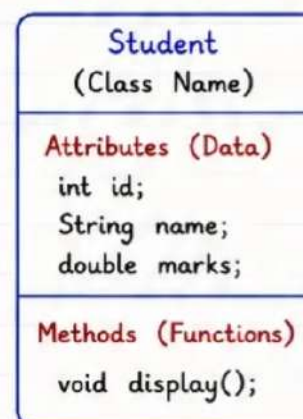
→ Attributes / Variables
(Data Members)

→ Methods / Functions
(Member Methods)

★ Example

```
class Student {
    // Attributes
    int id;
    String name;
    double marks;
    // Methods
    void display() {
        System.out.println("ID: " + id);
        System.out.println("Name: " + name);
        System.out.println("Marks: " + marks);
    }
}
```

Class Structure



→ Holds Data
(State)

→ Performs Actions
(Behavior)



Remember :

A class is just a **template**. No memory is allocated when a class is created. Memory is allocated only when objects are created from the class.

Example :

```
Student s1 = new Student();
// Object creation
```


★ ∴ OOPS CONCEPT FULL HANDBOOK ∴ ★

∴ 2. OBJECT ∴

★ What is an Object ?

An object is a real-world entity which is an instance of a class.
It has **state (data)** and **behavior (methods)**.

★ Key Points

- An object is **created** from a class.
- It occupies **memory**.
- Each object has its **own copy** of data.
- Objects can interact by **calling methods**.
- Multiple objects can be created from the **same class**.

★ Syntax (in Java)

```
ClassName objectName = new ClassName();
```

→ Class Name

→ Object Name (Reference)

→ new keyword

→ Constructor

★ Example

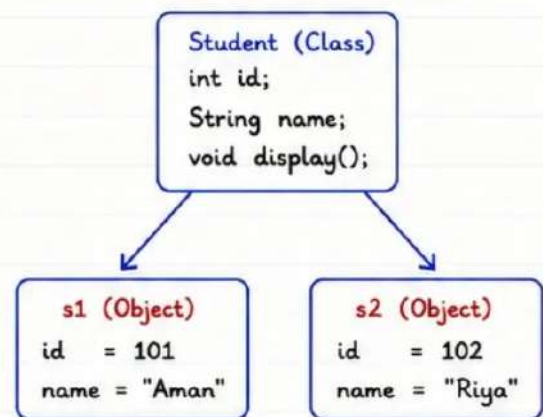
```
class Student {
    int id;
    String name;
    // Methods
    void display() {
        System.out.println(id + " - " + name);
    }
}

public class Main {
    public static void main(String[] args) {
        // Creating objects
        Student s1 = new Student();
        Student s2 = new Student();

        // Setting values
        s1.id = 101;
        s1.name = "Aman";
        s2.id = 102;
        s2.name = "Riya";

        // Calling method
        s1.display(); // Output: 101 - Aman
        s2.display(); // Output: 102 - Riya
    }
}
```

Object Diagram



Here, **s1** and **s2** are objects (instances) of class **Student**.
Each object has its **own data**.



Remember :

- Class is a blueprint.
- Object is a real entity created from the class.
- Many objects can be created from one class.

Example in Real Life :

Class : **Mobile**

Objects : **iphone, samsung, oneplus** (each mobile has its own details and behavior)

★ ⌋ OOPS CONCEPT FULL HANDBOOK ⌋ ★

⌋ 3. METHOD ⌋

★ What is a Method ?

A method is a block of code that performs a specific task.
It is defined inside a class and is used to define the behavior of objects.

★ Key Points

- A method is a function that belongs to a class.
- It is used to perform some operation.
- It can access and modify the data (attributes) of the class.
- Methods improve code modularity and reusability.
- Methods can return a value or may not return anything (void).

★ Syntax (in Java)

```
returnType methodName(parameters) {
    // method body
    // statements
    return value; // optional
}
```

Return type of method
(int, void, String, etc.)

Name of the method

Parameters passed to the method
(optional)

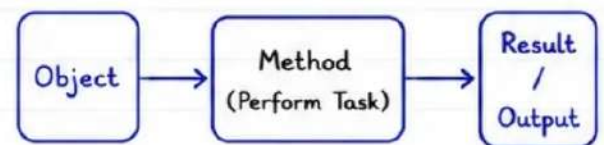
Method body

★ Example

```
class Calculator {
    int add(int a, int b) {
        int sum = a + b;
        return sum;
    }
    void showMessage() {
        System.out.println("Hello, Java!");
    }
}

public class Main {
    public static void main(String[] args) {
        Calculator c = new Calculator();
        int result = c.add(10, 20); // calling method with return
        System.out.println("Sum = " + result);
        c.showMessage(); // calling void method
    }
}
```

How Method Works ?



Object calls the method.

Method performs the task.

Method returns the result (if any).



Remember :

- Method is the behavior of an object.
- It is always defined inside a class.
- A method can return a value or nothing.
- Good method names describe what the method does.

Example in Real Life :

Think of a remote control.

Pressing a button (method) performs an action like changing channel, increasing volume, etc.

Each button has a specific function.

★ ≥ OOPS CONCEPT FULL HANDBOOK ≤ ★

≥ 4. CONSTRUCTOR ≤

★ What is a Constructor ?

A constructor is a special method in a class that is used to **initialize** objects. It is called **automatically** when an object is created.

★ Key Points

- Constructor name is **same** as class name.
- It has **no return type**, not even void.
- It is called **automatically** when object is created.
- Used to **initialize** object's data.
- A class can have **multiple** constructors (constructor overloading).
- If no constructor is written, Java provides a **default constructor**.

★ Syntax (in Java)

```
class ClassName {
    ClassName(parameters) {
        // initialization code
    }
}
```

→ Constructor name

→ Parameters (optional)

→ Constructor body

★ Example

```
class Student {
    int id;
    String name;
    // Constructor
    Student(int id, String name) {
        this.id = id;
        this.name = name;
    }
    void display() {
        System.out.println(id + " - " + name);
    }
}

public class Main {
    public static void main(String[] args) {
        Student s1 = new Student(101, "Aman");
        Student s2 = new Student(102, "Riya");
        s1.display(); // Output: 101 - Aman
        s2.display(); // Output: 102 - Riya
    }
}
```

How Constructor Works ?

new Student(101, "Aman")

↓
Constructor is called

↓
Object is created

↓
Data is initialized

Types of Constructors

- Default Constructor (no-arg)
- Parameterized Constructor
- Copy Constructor (via parameter)



Remember :

- Constructor runs **only once** when object is created.
- It helps to **initialize** object.
- Good constructor names describe what the object represents.

Example in Real Life :

When you register in any website, your **details** (name, email, password) are filled **automatically** in all required fields. That's how constructor works!



5. INHERITANCE

★ What is Inheritance ?

Inheritance is a mechanism where a new class (child/subclass) acquires the properties and behaviors (fields and methods) of an existing class (parent/superclass). It promotes code reusability and establishes an IS-A relationship.

★ Key Points

- The class whose members are inherited is called **superclass** (parent class).
- The class that inherits is called **subclass** (child class).
- The subclass can have its **own members** in addition to inherited members.
- Inheritance supports **code reusability** and **extensibility**.
- Java **does not** support multiple inheritance with classes (but supports it using interfaces).

★ Syntax (in Java)

```
class SuperClass {
    // fields and methods
}
class SubClass extends SuperClass {
    // own fields and methods
}
```

→ Parent / Superclass

→ Child / Subclass

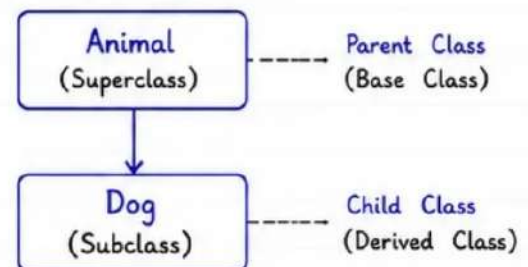
★ Example (in Java)

```
// Parent Class (Superclass)
class Animal {
    String color = "Brown";
    void eat() {
        System.out.println("Animal is eating");
    }
}

// Child Class (Subclass)
class Dog extends Animal {
    void bark() {
        System.out.println("Dog is barking");
    }
}

// Main Class
public class Main {
    public static void main(String[] args) {
        Dog d = new Dog();
        d.eat(); // inherited method
        d.bark(); // own method
        System.out.println(d.color); // inherited field
    }
}
```

Inheritance Diagram



How it Works ?

- ① Dog is a Animal. (IS-A Relationship)
- ② Dog inherits **color** and **eat()** from Animal.
- ③ Dog can use inherited members and also define its own members.

Benefits



- Code Reusability
- Easy Maintenance
- Extensibility
- Better Organization

Remember

- ✓ Use **extends** keyword to inherit.
- ✓ Private members are **not** inherited.
- ✓ Constructors are **not** inherited.
- ✓ Inheritance represents **IS-A** relationship.



6. POLYMORPHISM

★ What is Polymorphism ?

Polymorphism means "many forms". It allows one interface to be used for different types.

In Java, polymorphism is achieved by **Method Overloading** and **Method Overriding**.

★ Types of Polymorphism

1. Compile-time Polymorphism (Method Overloading)
2. Run-time Polymorphism (Method Overriding)

1. Compile-time Polymorphism (Method Overloading)

- Occurs in the **same class**.
- Same method name but different **parameter list** (number, type or order).
- It is resolved at **compile time**.

Example (in Java)

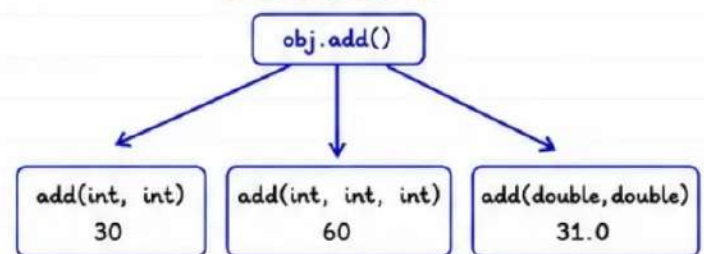
```
class Calculator {
    int add(int a, int b) {
        return a + b;
    }

    int add(int a, int b, int c) {
        return a + b + c;
    }

    double add(double a, double b) {
        return a + b;
    }
}

public class Main {
    public static void main(String[] args) {
        Calculator obj = new Calculator();
        System.out.println(obj.add(10, 20)); // 30
        System.out.println(obj.add(10, 20, 30)); // 60
        System.out.println(obj.add(10.5, 20.5)); // 31.0
    }
}
```

How it Works ?



The compiler decides which method to call based on the **arguments** passed.

2. Run-time Polymorphism (Method Overriding)

- Occurs in different classes using **inheritance**.
- Same method name, **same parameters**.
- It is resolved at **runtime**.

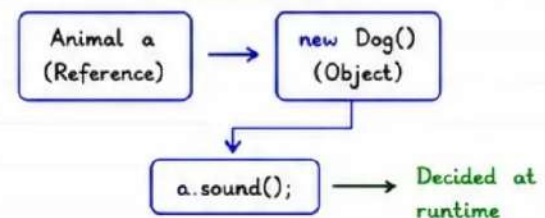
Example (in Java)

```
class Animal {
    void sound() {
        System.out.println("Animal makes sound");
    }
}

class Dog extends Animal {
    @Override
    void sound() {
        System.out.println("Dog barks");
    }
}

public class Main {
    public static void main(String[] args) {
        Animal a = new Dog(); // Reference of parent, object of child
        a.sound(); // Calls overridden method
    }
}
```

How it Works ?



JVM decides at **runtime** which method to call based on the **actual object**.

Key Difference

Feature	Method Overloading	Method Overriding
Type	Compile-time	Run-time
Where	Same class	Different class (Inheritance)
Parameters	Different parameter list	Same parameters
Binding	Static (early binding)	Dynamic (late binding)
Purpose	Increase readability	Achieve runtime polymorphism

Remember



- Overloading increases **readability**.
- Overriding achieves **runtime polymorphism**.
- Overloading is **static** (compile time).
- Overriding is **dynamic** (runtime).
- Both are key features of **OOPs** in Java.



7. ABSTRACTION

★ What is Abstraction ?

Abstraction means hiding the internal implementation details and showing only the essential features to the user. You focus on **what** an object does, not **how** it does it.

★ Key Points

- Hides complexity — exposes only the necessary functionality.
- Achieved using abstract classes (0–100%) and interfaces (100%).
- The **abstract** keyword is used for both classes and methods.
- An abstract method has only a declaration — **no body**.
- An abstract class **cannot be instantiated** (you cannot create its object).

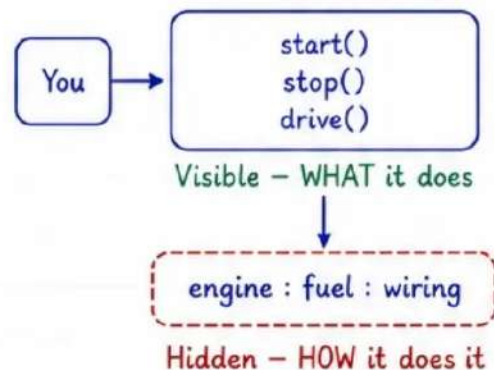
★ Syntax (in Java)

```
abstract class Shape {  
    abstract void area();    // abstract method (no body)  
    void info() {            // normal (concrete) method  
        System.out.println("This is a shape");  
    }  
}
```

★ Example (in Java)

```
abstract class Shape {  
    abstract double area();  
}  
  
class Circle extends Shape {  
    double r = 5;  
    double area() { return 3.14 * r * r; }  
}  
  
class Rectangle extends Shape {  
    double l = 4, w = 3;  
    double area() { return l * w; }  
}  
  
public class Main {  
    public static void main(String[] a) {  
        Shape s = new Circle();  
        System.out.println(s.area()); // 78.5  
    }  
}
```

How it Works ?



Abstract Class vs Interface

- **Abstract class** → abstract and concrete methods, uses **extends**.
- **Interface** → only abstract (+ default/static in Java 8+), uses **implements**.

Example in Real Life

Driving a car — you use the steering wheel, accelerator and brake **without knowing** the engine internals. Same idea: ATM & TV remote.



➤ 8. this & super KEYWORD ⚡

★ What are this & super ?

this refers to the **current object** of a class, while *super* refers to the **immediate parent (super)** class object. They remove ambiguity and let a child reuse parent members.

this → current object

- Refers to the **current** class instance.
- *this.x* = *x* resolves field vs parameter clashes.
- *this()* calls another constructor of the same class.
- Can pass the current object as an argument.

super → parent object

- *super.x* / *super.method()* access parent members.
- *super()* calls the parent class constructor.
- Lets a child reuse an overridden parent method.
- Always the **immediate** parent class.

★ Example (in Java)

```
class Animal {
    String type = "Animal";
    Animal() { System.out.println("Animal created"); }
    void sound() { System.out.println("Some sound"); }
}
class Dog extends Animal {
    String type = "Dog";
    Dog() {
        super(); // calls Animal()
        System.out.println(super.type + " → " + this.type);
    }
    void sound() {
        super.sound(); // parent version first
        System.out.println("Dog barks");
    }
}
```

Key Difference

this	super
refers to current object	refers to parent object
<i>this()</i> → same-class constructor	<i>super()</i> → parent constructor
<i>this.x</i> → current field	<i>super.x</i> → parent field

Remember

- ✓ *super()* / *this()* must be the **first** statement in a constructor.
- ✓ Use *super* when a child overrides but still needs the parent.
- ✓ If you write *super()* yourself, Java skips the auto one.

≡ 9. INTERFACE ≡

★ What is an Interface ?

An interface is a **blueprint** of a class that contains only abstract methods and constants. It specifies **what** a class must do, not **how** — giving **100% abstraction** and **multiple inheritance**.

★ Key Points

- A class uses the **implements** keyword to use an interface.
- All variables are **public static final** (constants) by default.
- All methods are **public abstract** by default (before Java 8).
- Java 8+ allows **default** & **static** methods; Java 9+ allows **private** methods.
- A class can implement **multiple interfaces** → **multiple inheritance**.

★ Syntax (in Java)

```
interface Animal {
    void sound();           // public abstract by default
}

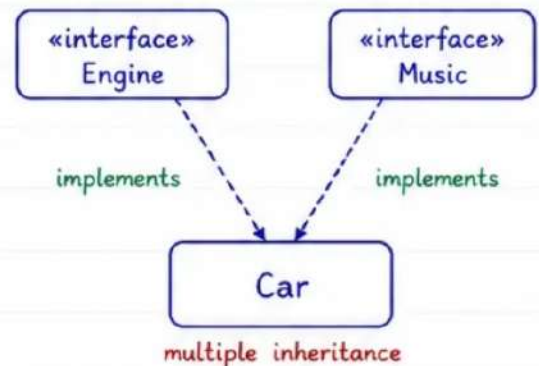
class Dog implements Animal {
    public void sound() {
        System.out.println("Dog barks");
    }
}
```

★ Example (in Java)

```
interface Engine { void start(); }
interface Music { void play(); }

class Car implements Engine, Music {
    public void start() {
        System.out.println("Engine starts");
    }
    public void play() {
        System.out.println("Music plays");
    }
}
```

Multiple Inheritance



Interface vs Abstract Class

Feature	Interface	Abstract Class
Methods	abstract (+default/static)	abstract + concrete
Variables	public static final (constants)	any type
Inheritance	multiple	single
Keyword	implements	extends

Example in Real Life

A USB port (or power socket) is an **interface** — any device that follows the **standard** can plug in, no matter how it is built inside.

★ ➤ OOPS CONCEPT FULL HANDBOOK ⚡ ★

➤ 10. ACCESS MODIFIERS ⚡

★ What are Access Modifiers ?

Access modifiers are keywords that set the **visibility (scope)** of classes, methods and variables — they decide **from where** a member can be accessed.

★ Key Points

- **private** → accessible only within the **same class**.
- **default** (no keyword) → accessible within the **same package**.
- **protected** → same package + **subclasses** (even in other packages).
- **public** → accessible from **everywhere**.
- Prefer the most restrictive level that works — good encapsulation.

Visibility Table

Modifier	Same Class	Same Package	Subclass	Everywhere
private	✓	✗	✗	✗
default	✓	✓	✗	✗
protected	✓	✓	✓	✗
public	✓	✓	✓	✓

★ Example (in Java)

```
class Account {
    private double balance; // inside class
    String owner;           // package
    protected int type;     // + subclass
    public int id;           // everywhere
}
```

Scope (widening out)

public — everywhere

protected — + subclass

default — same package

private

Remember

- ✓ **private** is the most restrictive, **public** the least.
- ✓ Keep fields **private**, expose via getters/setters.
- ✓ **protected** is mainly for inheritance.

Example in Real Life

Like rooms in a house: **private** = your locked bedroom, **default** = your family, **protected** = relatives with a key, **public** = the open street.



11. STATIC KEYWORD

★ What is the static Keyword ?

static means a member belongs to the *class itself* rather than to any object. A *single copy* is created and *shared* by every object of the class.

★ Key Points

- *static variable* → one *shared* copy for all objects (class variable).
- *static method* → called using the *class name*, no object needed.
- A *static method* can use *only static data* directly (no *this*).
- *static block* → runs *once* when the class is loaded into memory.
- *main()* is *static* so the JVM can run it without creating an object.

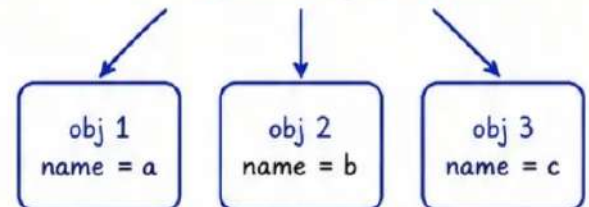
★ Example (in Java)

```
class Counter {  
    static int count = 0;    // shared by all  
    Counter() {  
        count++;            // shared copy  
    }  
    static void show() {  
        System.out.println("Objects = " + count);  
    }  
}  
  
public class Main {  
    public static void main(String[] a) {  
        new Counter();  
        new Counter();  
        new Counter();  
        Counter.show();    // Objects = 3  
    }  
}
```

Shared vs Instance

static int count = 3

one copy shared by all objects



each object keeps its own non-static data, but they all share the same static field.

Remember

- ✓ *static* = class level, non-static = object level.
- ✓ Access via *ClassName.member*.
- ✓ Loaded *once*, before any object is made.

Example in Real Life

A company name is the *same* for every *employee* (*static*), but each employee has their own name & ID (*instance*). Like a *shared* counter of created objects.



★ ≥ OOPS CONCEPT FULL HANDBOOK ≤ ★

≥ 12. ENCAPSULATION ≤

★ What is Encapsulation ?

Encapsulation is the mechanism of wrapping data (variables) and code (methods) together as a single unit and restricting direct access to some of the object's components. It helps in data hiding and protects the data from unauthorized access.

★ Key Points

- We achieve encapsulation by using access modifiers.
- Class variables should be **private**.
- Access to variables is provided through **public** getter and **setter** methods.
- It **protects** data from being modified by accident.
- It improves code **maintainability** and **flexibility**.

★ Syntax (in Java)

```
class ClassName {
    private dataType variable; // private variable

    // getter method
    public dataType getVariable() {
        return variable;
    }

    // setter method
    public void setVariable(dataType value) {
        variable = value;
    }
}
```

★ Example (in Java)

```
class Student {
    private int id;
    private String name;

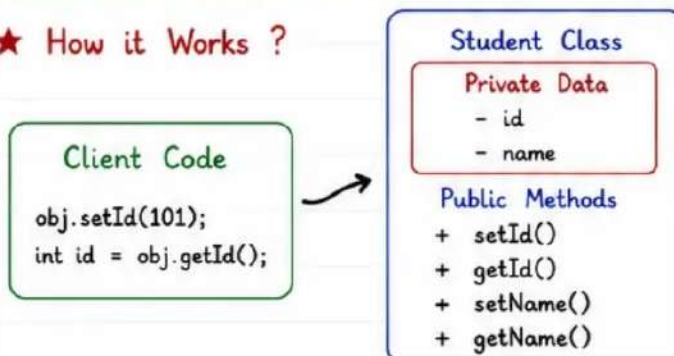
    // Getter for id
    public int getId() {
        return id;
    }

    // Setter for id
    public void setId(int id) {
        this.id = id;
    }

    // Getter for name
    public String getName() {
        return name;
    }

    // Setter for name
    public void setName(String name) {
        this.name = name;
    }
}
```

★ How it Works ?



Benefits

- ✓ Protects data from unauthorized access.
- ✓ Improves security.
- ✓ Allows controlled access to data.
- ✓ Makes code easy to maintain and modify.

Remember

- ✓ Make variables private.
- ✓ Provide public getter and setter methods.
- ✓ Access and modify data only through methods.

★ ≥ OOPS CONCEPT FULL HANDBOOK ≤ ★

≥ 9. SUMMARISE ALL 4 OOPS PILLARS ≤

OOPs is built on four main pillars. Together, they help us design robust, reusable and maintainable software.

Pillar	Meaning	Key Points	Example (in Java)
1. Encapsulation Wrapping data and methods together as a single unit and restricting direct access.	It protects the data and hides internal details using access modifiers .	- Achieved using private variables. - Access provided through public getter and setter methods. - Improves security and maintainability.	<pre>class Student { private int id; private String name; public int getId() { return id; } }</pre>
2. Abstraction Hiding implementation details and showing only essential features.	It focuses on what an object does instead of how it does it.	- Achieved using abstract classes and interfaces. - Reduces complexity. - Helps in defining contracts.	<pre>abstract class Shape { abstract void draw(); } class Circle extends Shape { void draw() { // implementation } }</pre>
3. Inheritance Acquiring properties and behaviors of one class into another class.	It promotes code reusability and establishes an IS-A relationship.	- Achieved using extends (class) and implements (interface). - Child class inherits fields and methods of parent class. - Supports method overriding.	<pre>class Animal { void eat() { } } class Dog extends Animal { void bark() { } }</pre>
4. Polymorphism Ability of an object to take many forms.	It allows one interface to be used for different types.	- Achieved through method overloading and method overriding. - Increases flexibility. - Supports compile-time and run-time polymorphism.	<pre>class Animal { void sound() { } } class Dog extends Animal { @Override void sound() { System.out.println("Bark"); } }</pre>

In Short

- ✓ Encapsulation protects data and restricts access.
- ✓ Abstraction hides details and shows only essential features.
- ✓ Inheritance allows code reusability through IS-A relationship.
- ✓ Polymorphism allows one interface to work with multiple forms.

Together, these four pillars:

- Make code reusable
- Reduce complexity
- Improve flexibility
- Increase security and maintainability